TOBB ETÜ
Ekonomi ve Teknoloji Üniversitesi

TOBB EKONOMİ VE TEKNOLOJİ ÜNİVERSİTESİ
Bilgisayar / Yapay Zekâ Mühendisliği
BIL 372 – Database Systems

Project Final Report

Hotel Chain, Conference, and Service Management System

Kaynak Kod ve Github linki: https://github.com/Metekilic0/hotel-management-system-database-project

İsim Soyisim	Mete KILIÇ
Numara	231401028
Akademik Dönem	2025–2026 Yaz Dönemi

Table Of Contents

1. Requirement Analysis

1.1 Project Objective

The system is a relational database supporting the daily operations of a multi-hotel luxury chain across four functional areas: Accommodation and Spaces, Clientele, Human Resources, and Reservations/Conference/Services. The mini-world description leaves several operational details unspecified; the assumptions below make those details concrete so that the conceptual, logical, and physical designs that follow are unambiguous.

1.2 Documented Business-Rule Assumptions

- A reservation groups one or more accommodation rooms under a single, shared date window (one check-in/check-out pair per ReservationID). Even though the stay dates are shared, each room line still stores its own agreed nightly rate, because room prices may legitimately change over time and a guest's rate should stay locked to what was agreed at booking time.
- Only CORPORATE_CUSTOMER accounts may create a CONFERENCE_BOOKING against a CONFERENCE_ROOM. This is enforced structurally (the booking's foreign key references the CORPORATE_CUSTOMER subtype table, which only contains corporate rows), not just by convention.
- Each EMPLOYEE reports to at most one supervisor, modeled as an optional (0..1) recursive relationship on EMPLOYEE. Only MANAGER-type employees are expected to have subordinates, but this is treated as a data convention rather than a hard structural constraint, to keep the hierarchy flexible.
- Every SERVICE_USAGE (a service charged to a guest's stay) must reference an existing, single RESERVATION and be fulfilled by exactly one EMPLOYEE; a service cannot be charged to a guest who has no active reservation.
- LoyaltyPoints only apply to INDIVIDUAL_GUEST customers; CORPORATE_CUSTOMER accounts do not accrue personal loyalty points, since bookings on their behalf are organizational rather than personal.
- A CONTRACT between a corporate customer and the hotel chain is optional context for a CONFERENCE_BOOKING (a corporate customer may hold an ad-hoc, one-off event booking without a standing contract).
- A MAINTENANCE_TICKET is opened against exactly one SPACE and assigned to exactly one TECHNICIAN for its entire lifecycle (no ticket hand-off/reassignment history is modeled in this project's scope).
- Room/space Status (AVAILABLE, OCCUPIED, MAINTENANCE, OUT_OF_SERVICE) is tracked uniformly at the SPACE (super-type) level, regardless of whether the space is an accommodation room or a conference room.

1.3 Major System Users and Information Needs

User	Information Needs
Receptionist	Room/space availability, guest identity and reservation details, check-in/check-out processing.
Housekeeper	Assigned zone and current room status per shift.
Technician	Open/assigned maintenance tickets and space location details.
Manager	Contracts under their oversight, team supervision, hotel-level revenue and occupancy analytics.
Corporate client contact	Their own contracts, conference bookings, and consolidated billing.
Chain-level analyst/executive	Cross-hotel revenue, occupancy, and service-usage analytics (via views).

1.4 Scope Boundaries — What the System Does NOT Manage

- Payment processing / credit-card or gateway integration (only charges and totals are recorded, not how they are settled).
- A loyalty-point redemption catalog or rewards engine (points are accumulated and stored, not redeemed).
- Full payroll, shift scheduling, or HR leave management beyond a basic ShiftType attribute.
- Marketing, CRM campaigns, or guest communication/notification workflows.
- Physical inventory of housekeeping/maintenance supplies and parts.
- Multi-currency conversion — all monetary amounts are assumed to be in a single base currency.

2. Conceptual Design — EER Diagram

The diagram below models the mini-world using three specialization hierarchies (SPACE, CUSTOMER, EMPLOYEE — exceeding the required minimum of two), one recursive relationship (SUPERVISES, on EMPLOYEE), one weak+associative entity used for the reservation – room many-to-many relationship (RESERVATION_ROOM), a second weak entity (MAINTENANCE_TICKET, existence-dependent on SPACE), a composite attribute (HOTEL.Address), a multivalued attribute (CUSTOMER.PhoneNumbers), and a derived attribute (RESERVATION.TotalCost).

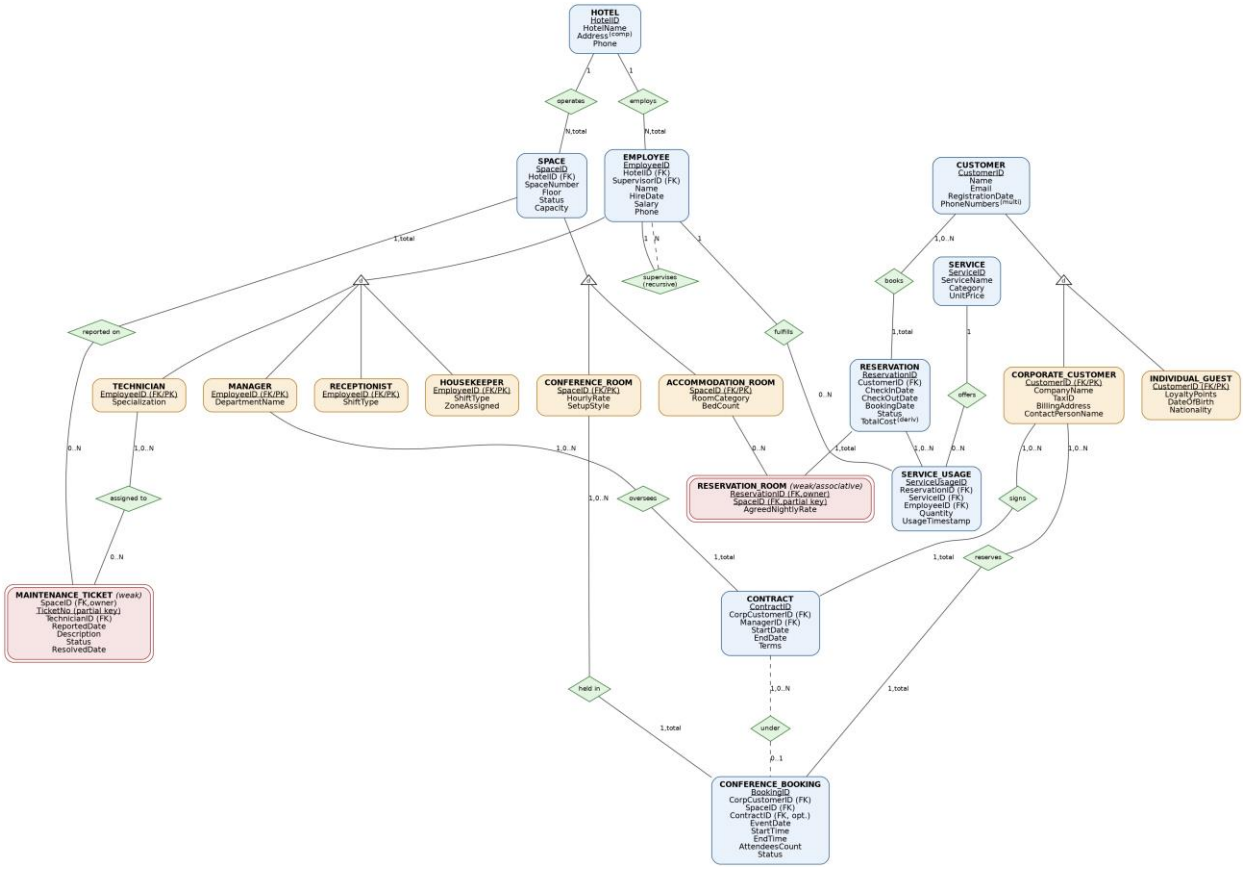

Figure 1. Conceptual EER diagram of the Hotel Chain, Conference, and Service Management System.

2.1 Specialization Hierarchies and Constraints

Hierarchy	Constraint	Justification
SPACE → {ACCOMMODATION_ROOM, CONFERENCE_ROOM}	Disjoint, Total	Every space is exactly one of the two kinds; a room cannot simultaneously be a guest room and a conference hall, and every space must be one or the other (no generic, un-typed spaces).
CUSTOMER → {INDIVIDUAL_GUEST, CORPORATE_CUSTOMER}	Disjoint, Total	A customer account is either a personal guest or a corporate account, never both, and every customer must fall into one of the two categories to be usable by the system.
EMPLOYEE → {RECEPTIONIST, HOUSEKEEPER, MANAGER, TECHNICIAN}	Disjoint, Total	Each staff member holds exactly one operational role in this project's scope; every hired employee must have a role to be schedulable/assignable.

2.2 Other Notable EER Constructs

Construct	Element	Notes
Associative (M:N) entity	RESERVATION_ROOM	Resolves the many-to-many relationship between RESERVATION and ACCOMMODATION_ROOM; carries the AgreedNightlyRate attribute on the relationship itself.
Weak entity	RESERVATION_ROOM / MAINTENANCE_TICKET	Both are existence-dependent on an owner entity (RESERVATION and SPACE respectively) and use a partial key (SpaceID, TicketNo).
Recursive relationship	SUPERVISES (on EMPLOYEE)	Models that a manager (an EMPLOYEE) supervises other EMPLOYEEs, via a self-referencing SupervisorID foreign key.
Composite attribute	HOTEL.Address	Decomposed into Street, City, Country, PostalCode.
Multivalued attribute	CUSTOMER.PhoneNumbers	A customer may have more than one contact number; mapped to CUSTOMER_PHONE.
Derived attribute	RESERVATION.TotalCost	Computed from room-night charges (RESERVATION_ROOM) plus service charges (SERVICE_USAGE); kept up to date by a trigger rather than recalculated by every reader.

3. Mapping the EER Model to the Relational Model

3.1 Mapping Strategy

Regular (non-weak) entities map one-to-one to relations, using their key as primary key. The three disjoint/total specialization hierarchies are each mapped using the classic "one table per subclass, sharing the superclass's primary key as both primary key and foreign key" option: this avoids nullable columns for attributes that only apply to one subtype, and keeps every subtype table's existence tied to exactly one supertype row via ON DELETE CASCADE. Weak entities (RESERVATION_ROOM, MAINTENANCE_TICKET) map to relations whose primary key is the composite of the owner's key plus the partial key. The multivalued attribute CUSTOMER.PhoneNumbers maps to its own relation (CUSTOMER_PHONE) keyed by (CustomerID, PhoneNumber). The composite attribute HOTEL.Address is simply flattened into its component columns (Street, City, Country, PostalCode) directly inside HOTEL. The derived attribute RESERVATION.TotalCost is materialized as a stored column, maintained by triggers, to avoid recomputing it on every read.

3.2 Relational Schemas (Primary/Foreign Keys and Referential Actions)

Relation	Primary Key	Foreign Key(s)	Referential Action & Justification
HOTEL	HotelID	—	—
SPACE	SpaceID	HotelID → HOTEL	CASCADE (a space cannot outlive its hotel)
ACCOMMODATION_ROOM	SpaceID	SpaceID → SPACE	CASCADE (subtype row dies with its supertype row)
CONFERENCE_ROOM	SpaceID	SpaceID → SPACE	CASCADE
CUSTOMER	CustomerID	—	—
CUSTOMER_PHONE	(CustomerID, PhoneNumber)	CustomerID → CUSTOMER	CASCADE
INDIVIDUAL_GUEST	CustomerID	CustomerID → CUSTOMER	CASCADE
CORPORATE_CUSTOMER	CustomerID	CustomerID → CUSTOMER	CASCADE
EMPLOYEE	EmployeeID	HotelID → HOTEL; SupervisorID → EMPLOYEE (self)	RESTRICT on HotelID; SET NULL on SupervisorID (losing a supervisor should not delete subordinates)
RECEPTIONIST / HOUSEKEEPER / MANAGER / TECHNICIAN	EmployeeID	EmployeeID → EMPLOYEE	CASCADE
RESERVATION	ReservationID	CustomerID → CUSTOMER	RESTRICT (a customer with reservation history cannot be silently deleted)
RESERVATION_ROOM (weak)	(ReservationID, SpaceID)	ReservationID → RESERVATION; SpaceID → ACCOMMODATION_ROOM	CASCADE on ReservationID (weak entity dies with owner); RESTRICT on SpaceID (a booked room cannot be deleted from inventory)
SERVICE	ServiceID	—	—

Relation	Primary Key	Foreign Key(s)	Referential Action & Justification
SERVICE_USAGE	ServiceUsageID	ReservationID → RESERVATION; ServiceID → SERVICE; EmployeeID → EMPLOYEE	CASCADE on ReservationID; RESTRICT on ServiceID/EmployeeID
CONTRACT	ContractID	CorporateCustomerID → CORPORATE_CUSTOMER; ManagerID → MANAGER	CASCADE on CorporateCustomerID; RESTRICT on ManagerID
CONFERENCE_BOOKING	BookingID	CorporateCustomerID → CORPORATE_CUSTOMER; SpaceID → CONFERENCE_ROOM; ContractID → CONTRACT	RESTRICT on CorporateCustomerID/SpaceID; SET NULL on ContractID (a booking may outlive/detach from a closed contract)
MAINTENANCE_TICKET (weak)	(SpaceID, TicketNo)	SpaceID → SPACE; TechnicianID → TECHNICIAN	CASCADE on SpaceID (weak entity dies with owner); RESTRICT on TechnicianID

3.3 Candidate / Alternate Keys

- HOTEL: HotelName is an alternate (candidate) key in addition to the surrogate HotelID.
- SPACE: (HotelID, SpaceNumber) is an alternate key — a room/hall number is only guaranteed unique within its own hotel.
- CUSTOMER: Email is an alternate key.
- CORPORATE_CUSTOMER: TaxID is an alternate key.
- SERVICE: ServiceName is an alternate key.

3.4 Specialization Mapping Notes

All three hierarchies are disjoint and total, so the chosen “shared-PK subtype table” mapping is lossless and never requires NULL-padded attributes from unrelated subtypes. Totality is not fully enforceable by foreign keys alone (a SPACE row could in principle exist with no matching ACCOMMODATION_ROOM/CONFERENCE_ROOM row); the SpaceType/CustomerType/EmployeeType discriminator columns record the intended subtype for query convenience, and application-level/insert discipline (documented here) is relied upon to guarantee exactly one matching subtype row is always inserted together with its supertype row. A stricter enforcement (e.g., a deferred constraint trigger checking discriminator-vs-subtype-table consistency) is noted as a possible extension beyond this project's required scope.

3.5 Business Rules Represented in the Relational Design

- Overlap-free room booking: enforced by the mandatory trigger on RESERVATION_ROOM (Section 6).
- Only corporate customers may hold conference bookings: enforced structurally via the FK CONFERENCE_BOOKING.CorporateCustomerID → CORPORATE_CUSTOMER.
- Check-out must be after check-in / event end after start: enforced with CHECK constraints on RESERVATION and CONFERENCE_BOOKING.
- A manager cannot supervise themselves: enforced with a CHECK constraint on EMPLOYEE.

- Loyalty points are non-negative and salaries/prices are non-negative: enforced with CHECK constraints throughout.

4. Normalization Task —UNNORMALIZED_GUEST_INVOICE

4.0 Setting Up the Problem

The raw extract stores one row per (InvoiceNo, RoomNo, ServiceUsageID) combination. This grain is itself the root problem: a reservation may legitimately involve several rooms and, independently, several logged service usages, but flattening both repeating facts into a single row produces a spurious cross-product between rooms and services that have no real relationship to each other (a classic "connection trap"). We therefore begin 1NF with the all-attribute-covering key:

```
K = { InvoiceNo, RoomNo, ServiceUsageID }
(HotelID accompanies RoomNo only because room numbers are unique
within a hotel, not globally — see (HotelID,RoomNo) → RoomType,RoomRate)
```

Additional reasonable dependency assumed (semantics unaffected): ServiceUsageID → ReservationID, i.e. a logged service usage belongs to exactly one reservation/stay — this mirrors real-world semantics and is used below to avoid an artificial room/service junction that the given dependencies do not actually support.

4.1 First Normal Form (1NF)

All attributes in the source relation are already atomic (single-valued, indivisible), and the row grain above gives a well-defined composite key, so the relation qualifies as 1NF. It is, however, full of partial and transitive dependencies, which we remove next.

4.2 Second Normal Form (2NF) — Removing Partial Dependencies

A partial dependency exists when a non-prime attribute depends on a proper subset of a composite candidate key. Checking each given FD against K = {InvoiceNo, RoomNo, ServiceUsageID}:

- InvoiceNo → InvoiceDate, ReservationID, TotalAmount — PARTIAL (depends only on InvoiceNo).
- (HotelID, RoomNo) → RoomType, RoomRate — PARTIAL (does not involve InvoiceNo or ServiceUsageID).
- ServiceUsageID → ServiceID, ServiceQuantity, ServiceUnitPrice, ServiceTimestamp, EmployeeID — PARTIAL (depends only on ServiceUsageID).
- ReservationID → GuestID, HotelID, CheckInDate, CheckOutDate — transitive via InvoiceNo (handled in 3NF, Section 4.3).
- GuestID → GuestName, GuestLoyaltyPoints — transitive (Section 4.3).
- HotelID → HotelName, HotelAddress — transitive (Section 4.3).
- ServiceID → ServiceDescription — transitive (Section 4.3).
- EmployeeID → EmployeeName, EmployeeRole — transitive (Section 4.3).

Removing the three PARTIAL dependencies yields the following 2NF decomposition (no attribute is left that depends on the full three-part key, confirming the original grain was an unwarranted flattening of two independent facts):

```
R1  INVOICE_2NF (InvoiceNo, InvoiceDate, ReservationID, TotalAmount)
R2  ROOM        (HotelID, RoomNo, RoomType, RoomRate)
R3  SVC_USAGE_2NF(ServiceUsageID, ReservationID, ServiceID, ServiceQuantity,
                  ServiceUnitPrice, ServiceTimestamp, EmployeeID)
R4  RESERVATION_ROOM (ReservationID, RoomNo, HotelID)   -- the true M:N linkage
```

4.3 Third Normal Form (3NF) — Removing Transitive Dependencies

A transitive dependency exists when a non-prime attribute depends on another non-prime attribute rather than directly on the key. Applying each transitive FD identified above to R1 and R3:

```
From R1 (key InvoiceNo):
  ReservationID -> GuestID, HotelID, CheckInDate, CheckOutDate   (transitive)
     => extract RESERVATION(ReservationID, GuestID, HotelID, CheckInDate,
CheckOutDate)
  GuestID -> GuestName, GuestLoyaltyPoints                        (transitive, via
RESERVATION)
     => extract GUEST(GuestID, GuestName, GuestLoyaltyPoints)
  HotelID -> HotelName, HotelAddress                              (transitive, via
RESERVATION)
     => extract HOTEL(HotelID, HotelName, HotelAddress)
  remainder: INVOICE(InvoiceNo, InvoiceDate, ReservationID(FK), TotalAmount)

From R3 (key ServiceUsageID):
  ServiceID -> ServiceDescription                                 (transitive)
     => extract SERVICE(ServiceID, ServiceDescription)
  EmployeeID -> EmployeeName, EmployeeRole                        (transitive)
     => extract EMPLOYEE(EmployeeID, EmployeeName, EmployeeRole)
  remainder: SERVICE_USAGE(ServiceUsageID, ReservationID(FK), ServiceID(FK),
                           ServiceQuantity, ServiceUnitPrice, ServiceTimestamp,
                           EmployeeID(FK))
```

4.4 Final Decomposition (3NF achieved)

Relation	Primary Key	Remaining Attributes
HOTEL	HotelID	HotelName, HotelAddress
ROOM	(HotelID, RoomNo)	RoomType, RoomRate [FK HotelID → HOTEL]
GUEST	GuestID	GuestName, GuestLoyaltyPoints
RESERVATION	ReservationID	GuestID(FK), HotelID(FK), CheckInDate, CheckOutDate
RESERVATION_ROOM	(ReservationID, RoomNo)	HotelID [FK ReservationID → RESERVATION; FK (HotelID,RoomNo) → ROOM]
INVOICE	InvoiceNo	InvoiceDate, ReservationID(FK), TotalAmount
SERVICE	ServiceID	ServiceDescription
EMPLOYEE	EmployeeID	EmployeeName, EmployeeRole
SERVICE_USAGE	ServiceUsageID	ReservationID(FK), ServiceID(FK), ServiceQuantity, ServiceUnitPrice, ServiceTimestamp, EmployeeID(FK)

4.5 Boyce-Codd Normal Form (BCNF) Check

A relation is in BCNF if, for every non-trivial FD X → Y that holds, X is a superkey. Walking through each relation above:

- HOTEL: only determinant is HotelID, which is the PK → BCNF.
- ROOM: only determinant is (HotelID, RoomNo), the PK (room numbers repeat across different hotels, so RoomNo alone is not a determinant) → BCNF.

- GUEST: only determinant is GuestID, the PK → BCNF.
- RESERVATION: only determinant is ReservationID, the PK → BCNF.
- RESERVATION_ROOM: only determinant is (ReservationID, RoomNo), the PK → BCNF.
- INVOICE: only determinant is InvoiceNo, the PK → BCNF.
- SERVICE: only determinant is ServiceID, the PK → BCNF.
- EMPLOYEE: only determinant is EmployeeID, the PK → BCNF.
- SERVICE_USAGE: only determinant is ServiceUsageID, the PK → BCNF.

Every relation is therefore already in BCNF once transitive dependencies are removed — no further decomposition (and no accompanying lossy-join / dependency-preservation trade-off) is required for this exercise.

Note on the main project schema

This normalization exercise operates on the legacy, denormalized log exactly as given in the assignment. Its resulting INVOICE/ROOM/RESERVATION_ROOM relations are intentionally simpler than the main operational schema in Section 3 (for example, the legacy log has no per-reservation "agreed nightly rate," only a room's current RoomRate). This is a deliberate observation: the legacy log would need to be reconciled with the richer operational design before being merged into the live system, since one is a snapshot of "current price" and the other preserves historical, agreed pricing per stay.

5. Relational Algebra Verification

The following formal relational algebra expressions are written against the main operational schema (Section 3) and correspond directly to Queries Q1, Q2, Q4, Q7 and Q8 in the accompanying SQL script, so their results can be cross-checked against the executed SQL output (Section 7).

5.1 Expression 1 — Selection + Projection (→ Q1)

Names and loyalty points of individual guests qualifying as VIP (more than 500 loyalty points):

```
π CustomerName, LoyaltyPoints ( σ LoyaltyPoints > 500 ( CUSTOMER ⋈ INDIVIDUAL_GUEST
) )
```

5.2 Expression 2 — Multi-way Join + Projection (→ Q2)

Guest name, hotel name, and room number for every reservation in the system:

```
π CustomerName, HotelName, SpaceNumber, CheckInDate, CheckOutDate (
    RESERVATION ⋈ CUSTOMER ⋈ RESERVATION_ROOM ⋈ SPACE ⋈ HOTEL
)
```

5.3 Expression 3 — Grouping / Aggregation, extended RA (→ Q4)

Average attendee count and total revenue per conference SetupStyle, using the extended relational-algebra grouping operator ɢ (script-G):

```
CONFERENCE_ROOM ⋈ CONFERENCE_BOOKING
  ɢ SetupStyle ɢ AVG(AttendeesCount) → AvgAttendees,
                 SUM(HourlyRate × DurationHours) → TotalRevenue
```

5.4 Expression 4 — Set Difference / Anti-join (→ Q7)

Contracts that were signed but have never been used for a single conference booking ("contract lifecycle risk"):

```
AllContracts   ←  π ContractID ( CONTRACT )
UsedContracts  ←  π ContractID ( σ ContractID IS NOT NULL ( CONFERENCE_BOOKING ) )

UnusedContracts ←  AllContracts − UsedContracts

Result ←  π CompanyName ( UnusedContracts ⋈ CONTRACT ⋈ CORPORATE_CUSTOMER )
```

5.5 Expression 5 — Relational Division (→ Q8)

Guests who have used every service category the hotel chain offers, at least once (a textbook division: the set of (guest, category) pairs actually used, divided by the set of all categories):

```
Used        ←  π CustomerID, Category (
                 CUSTOMER ⋈ RESERVATION ⋈ SERVICE_USAGE ⋈ SERVICE
               )
AllCategories ←  π Category ( SERVICE )

Qualifying  ←  Used ÷ AllCategories

Result      ←  π CustomerName ( Qualifying ⋈ CUSTOMER )
```

Recall (division definition): Used ÷ AllCategories = { t[CustomerID] | t ∈ Used ∧ ∀ c ∈ AllCategories, (t[CustomerID], c) ∈ Used }. With the supplied test data this evaluates to a single guest, "Elif Yildiz", who has logged at least one usage in all four service categories

(FOOD_BEVERAGE, WELLNESS, TRANSPORT, HOUSEKEEPING) — confirmed against the executed Q8 output in Section 7.

6. Physical Implementation — Constraints, Views, Triggers

The complete, executable DDL (including all CHECK constraints, indexes, test data inserts, views, and trigger definitions) is provided in the accompanying file hotel_project.sql. It has been validated end-to-end against a live PostgreSQL 16 instance — both during development and by a second, independent execution — and every statement executes without error, with every query producing the results shown in Section 7. This section explains the non-obvious design choices.

6.1 Constraint Highlights

- CHECK (CheckOutDate > CheckInDate) on RESERVATION and CHECK (EndTime > StartTime) on CONFERENCE_BOOKING prevent zero-length or backwards date/time windows.
- CHECK (SupervisorID IS NULL OR SupervisorID <> EmployeeID) on EMPLOYEE prevents an employee from supervising themselves.
- UNIQUE (HotelID, SpaceNumber) on SPACE enforces that room/hall numbers only need to be unique within a hotel, matching the BCNF justification in Section 4.5.
- All monetary and count attributes (Salary, UnitPrice, LoyaltyPoints, Capacity, Quantity, etc.) carry CHECK constraints preventing negative or zero values where zero is nonsensical.

6.2 Views

Three views are provided, covering both the “protect baseline infrastructure text fields” and “serve analytical platforms” requirements:

- v_employee_directory — exposes employee name, role, hotel, hire date and supervisor name, while hiding Salary and raw Phone (sensitive HR fields).
- v_hotel_revenue_summary — an analytical view aggregating room-night revenue and service revenue per hotel, for chain-level reporting.
- v_room_availability — a human-readable view joining SPACE with its subtype (accommodation or conference) and hotel name, hiding raw internal foreign-key identifiers behind names.

6.3 Triggers

Three trigger functions are implemented in total: the mandatory overlap-prevention trigger, the derived-attribute maintenance trigger, and a third trigger added specifically to close a gap discovered by re-examining the mandatory trigger's coverage (Section 6.3.3).

6.3.1 Overlap Prevention Trigger (mandatory)

fn_prevent_room_overlap() fires BEFORE INSERT OR UPDATE on RESERVATION_ROOM (the room-reservation linkage/associative table). For the room (SpaceID) being booked, it looks up the owning reservation's CheckInDate/CheckOutDate, then checks every other non-cancelled reservation of the same room for a temporally overlapping window, using the standard half-open interval overlap test a1 < b2 AND b1 < a2. If a conflict is found, the trigger raises an exception and the transaction is rejected. This was

verified twice, independently, against the live database: attempting to book Room 101 (SpaceID 1) for 2026-07-02..2026-07-03, which overlaps the existing Reservation #1 (2026-07-01..2026-07-04), is correctly rejected both times with the error “Overlap detected: SpaceID 1 is already reserved for an overlapping date range (2026-07-02,2026-07-03).”.

6.3.2 Derived-Attribute Maintenance Trigger (additional)

fn_recalculate_reservation_total() fires AFTER INSERT/UPDATE/DELETE on both RESERVATION_ROOM and SERVICE_USAGE. It recomputes RESERVATION.TotalCost as the sum of (AgreedNightlyRate × nights) across all of the reservation's room lines, plus (Quantity × UnitPrice) across all of its logged service usages, and writes the result back into RESERVATION. This keeps the EER model's derived attribute (Section 2.2) continuously consistent without requiring every reader to recompute it, and was chosen specifically because it ties directly back to a concept introduced in the conceptual design. Its effect is directly visible in Query Q5 (Section 7), whose AvgCost figures are read straight from this maintained column.

6.3.3 Overlap-Check Gap Closure Trigger (additional)

The mandatory trigger in 6.3.1 only fires on RESERVATION_ROOM. It was identified during review that a reservation's date window can also be changed directly on RESERVATION itself (UPDATE RESERVATION SET CheckInDate = ... / CheckOutDate = ...) without touching RESERVATION_ROOM at all — a path the mandatory trigger cannot see, since it never fires. Left unaddressed, this would let a date edit silently create an overlapping booking for a room already linked to that reservation. fn_prevent_overlap_on_reservation_dates() closes this gap: it fires BEFORE UPDATE OF CheckInDate, CheckOutDate ON RESERVATION and, whenever either date actually changes, re-checks every room already linked to that reservation (via RESERVATION_ROOM) against every other non-cancelled reservation of the same room(s), using the identical overlap test as 6.3.1.

This was independently verified against the live database in two scenarios: (1) updating Reservation #2's (Room 102) dates to 2026-09-11..2026-09-13, which overlaps Reservation #15's existing booking of the same room, was correctly rejected with “Overlap detected: changing ReservationID 2 to (2026-09-11,2026-09-13) would conflict with another reservation of one of its already-booked rooms.”; (2) a legitimate, non-conflicting shift of the same reservation's dates to 2026-07-06..2026-07-08 succeeded normally (UPDATE 1, confirmed by re-selecting the row), demonstrating that the trigger does not produce false positives.

7. Sample Queries and Verified Results

All eight queries below were designed to exercise distinctive parts of the schema (the recursive supervision hierarchy, the contract–conference relationship, the service-category division) rather than generic filters, and were executed twice against the populated PostgreSQL 16 database — once during development and once independently — with identical results both times. The full SQL text is in hotel_project.sql, Section 6.

#	Type	Purpose	Verified Result
Q1	Selection + Projection	VIP individual guests (LoyaltyPoints > 500) with a computed Gold/Platinum tier.	3 rows — Emre Sahin (1200, Platinum), Nihan Aktas (990, Gold), Elif Yildiz (850, Gold).
Q2	Three/multi-table join	Guest, hotel, and room number for every reservation.	18 rows (one per room line across all 15 reservations, including the 3 multi-room ones).
Q3	Outer join (recursive self-join)	Every manager with their supervised team size (0 included), via EMPLOYEE joined to itself on SupervisorID.	2 rows — Mehmet Demir (Aurora Bosphorus Istanbul, team of 5), Ayse Kaya (Aurora Grand Ankara, team of 3).
Q4	Aggregate + GROUP BY	Average attendees and total revenue per conference SetupStyle.	3 rows — BANQUET (avg 56.7, revenue 2640), THEATER (avg 67.5, revenue 2550), BOARDROOM (avg 17.5, revenue 720).
Q5	GROUP BY + HAVING	Guests with more than 1 non-cancelled reservation AND an average reservation cost above 300.	3 rows — Zeynep Celik (2 res., avg 578), Emre Sahin (2 res., avg 492), Elif Yildiz (2 res., avg 320.5).
Q6	Nested (sub)query	Corporate customers whose conference bookings drew more attendees than the average across all bookings.	3 rows — Mavi Turizm Organizasyon (100), Anadolu Insaat Holding (75), TechNova Yazilim A.S. (60).
Q7	Correlated NOT EXISTS	“Contract lifecycle risk”: corporate customers holding a contract under which no conference has ever been booked.	1 row — GlobalTrade Lojistik A.S., ContractID 6 (started 2026-07-01, open-ended, no bookings yet).
Q8	Relational division	Guests who have used every one of the 4 service categories at least once.	1 row — Elif Yildiz (only guest with logged FOOD_BEVERAGE, WELLNESS, TRANSPORT and HOUSEKEEPING usage).

Note on Q5's threshold: 300 was chosen after inspecting the populated data's per-guest average reservation cost (which ranges from about 166 to 578 across repeat guests); it cleanly separates the two highest-spending repeat guests from the rest while still leaving room for a third guest to qualify once service charges are included, giving a meaningful (non-trivial) 3-row result rather than an all-or-nothing split.

7.1 Sample Queries and Verified Results Examples

Q2:

```sql
-- Q2. Three-table join:
-- Show, for every reservation, the guest's name, the hotel name, and the room number booked.
SELECT c.CustomerName, h.HotelName, sp.SpaceNumber, r.CheckInDate, r.CheckOutDate
FROM RESERVATION r
JOIN CUSTOMER c ON c.CustomerID = r.CustomerID
JOIN RESERVATION_ROOM rr ON rr.ReservationID = r.ReservationID
JOIN SPACE sp ON sp.SpaceID = rr.SpaceID
JOIN HOTEL h ON h.HotelID = sp.HotelID
ORDER BY r.ReservationID;
```

customer(+) 1 | customer(+) 1 (2) | employee(+) 1 (3) | conference_room 1 (4) | customer 1 (5) | corporate_customer(+) 1 (6) | corporate_customer(+) 1 (7)

Enter a SQL expression to filter results (use Ctrl+Space)

	customername	hotelname	spacenumber	checkindate	checkoutdate
1	Elif Yildiz	Aurora Grand Ankara	101	2026-07-01	2026-07-04
2	Deniz Aydin	Aurora Grand Ankara	102	2026-07-05	2026-07-07
3	Zeynep Celik	Aurora Grand Ankara	201	2026-07-10	2026-07-12
4	Emre Sahin	Aurora Grand Ankara	101	2026-06-01	2026-06-05
5	Emre Sahin	Aurora Grand Ankara	102	2026-06-01	2026-06-05
6	Burak Ozdemir	Aurora Grand Ankara	301	2026-06-10	2026-06-12
7	Selin Aksoy	Aurora Bosphorus Istan	101	2026-07-15	2026-07-18
8	Kerem Polat	Aurora Bosphorus Istan	102	2026-07-20	2026-07-22
9	Nihan Aktas	Aurora Bosphorus Istan	101	2026-07-25	2026-07-28
10	Nihan Aktas	Aurora Bosphorus Istan	102	2026-07-25	2026-07-28
11	Elif Yildiz	Aurora Bosphorus Istan	202	2026-08-01	2026-08-03
12	Deniz Aydin	Aurora Bosphorus Istan	302	2026-08-05	2026-08-06
13	Zeynep Celik	Aurora Riviera Antalya	101	2026-08-10	2026-08-14
14	Zeynep Celik	Aurora Riviera Antalya	201	2026-08-10	2026-08-14
15	Emre Sahin	Aurora Grand Ankara	201	2026-05-01	2026-05-03
16	Burak Ozdemir	Aurora Grand Ankara	301	2026-05-15	2026-05-17
17	Selin Aksoy	Aurora Grand Ankara	101	2026-09-01	2026-09-03
18	Kerem Polat	Aurora Grand Ankara	102	2026-09-10	2026-09-12

Q7:

```sql
-- Q7. Correlated NOT EXISTS:
-- "Contract lifecycle risk": corporate customers who hold a CONTRACT under
-- which NOT A SINGLE conference booking has ever been made.
SELECT cc.CompanyName, ct.ContractID, ct.StartDate, ct.EndDate
FROM CONTRACT ct
JOIN CORPORATE_CUSTOMER cc ON cc.CustomerID = ct.CorporateCustomerID
WHERE NOT EXISTS (
    SELECT 1 FROM CONFERENCE_BOOKING cb
    WHERE cb.ContractID = ct.ContractID AND cb.Status <> 'CANCELLED'
)
ORDER BY ct.StartDate;
```

tomer(+) 1 | employee(+) 1 (3) | corporate_customer(+) 1 (6) | corporate_customer(+) 1 (7) | customer 1 (8)

Enter a SQL expression to filter results (use Ctrl+Space)

companyname	contractid	startdate	enddate
GlobalTrade Lojistik A.S.	6	2026-07-01	[NULL]

Q6:

```sql
-- Q6. Nested query:
-- Corporate customers whose conference bookings drew more attendees than the
-- average AttendeesCount across all (non-cancelled) conference bookings.
SELECT DISTINCT cc.CompanyName, cb.BookingID, cb.AttendeesCount
FROM CONFERENCE_BOOKING cb
JOIN CORPORATE_CUSTOMER cc ON cc.CustomerID = cb.CorporateCustomerID
WHERE cb.Status <> 'CANCELLED'
  AND cb.AttendeesCount > (
        SELECT AVG(AttendeesCount) FROM CONFERENCE_BOOKING WHERE Status <> 'CANCELLED'
      )
ORDER BY cb.AttendeesCount DESC;
```

tomer(+) 1 | employee(+) 1 (3) | corporate_customer(+) 1 (6) | corporate_customer(+) 1 (7) | customer 1 (8)

Enter a SQL expression to filter results (use Ctrl+Space)

companyname	bookingid	attendeescount
Mavi Turizm Organizasyon	4	100
Anadolu Insaat Holding	3	75
TechNova Yazilim A.S.	1	60

8. Test Data Summary

Entity	Count	Notes
Hotels	3	Aurora Grand Ankara, Aurora Bosphorus Istanbul, Aurora Riviera Antalya.
Spaces	13	10 accommodation rooms + 3 conference rooms (at least one conference room per hotel).
Employees	10	3 Receptionists, 3 Housekeepers, 2 Managers, 2 Technicians, distributed across all 3 hotels.
Individual guests	8	Loyalty points ranging from 0 to 1200.
Corporate customers	5	Linked to 6 CONTRACT rows overseen by MANAGERs (GlobalTrade Lojistik holds two contracts, one of them deliberately unused, to exercise Q7).
Reservations	15	Includes 3 multi-room reservations under a single ReservationID (IDs 4, 8, 11).
Conference bookings	7	Distributed across THEATER, BANQUET and BOARDROOM setup styles to exercise Q4's grouping.
Services / service usages	4 services, 15 usage logs	In-Room Dining, Spa Session, Airport Transit, Laundry Service; one guest (Elif Yildiz) deliberately spans all 4 categories to exercise Q8's division.
Maintenance tickets	3	2 COMPLETED, 1 OPEN.

9. Generative-AI Usage Declaration

AI was utilized as a collaborative tool in the following areas:

- **Normalization (Section 4):** Assisting in formatting and structuring the step-by-step decomposition from 1NF up to BCNF.
- **Conceptual EER Design:** Providing guidance on the correct notation and structural logic for associative entities, as well as helping to formulate some of the specialization constraints.
- **Relational Algebra:** Assisting in drafting some of the formal relational algebra expressions.

It is important to state that the AI was not used to autonomously generate the project. I actively directed the prompts, and all AI-provided suggestions were strictly reviewed, modified, and verified by me before inclusion. I independently cross-checked the normalization proofs against course principles, manually adjusted the EER constraints to ensure they perfectly matched the defined business rules, and rigorously validated the relational algebra logic. The final architectural decisions, the integration of the components, and the end-to-end testing on the PostgreSQL instance are entirely my own work.